LAB — K8s Argo CD (GitOps simple)

Table of Contents

LAB — K8s Argo CD (GitOps simple) 1

1 Objectif 2

2 Architecture 2

3 PRÉREQUIS 2

4 STEP 0 — Verifier Kubernetes 3

---

5 STEP 1 — Installer Argo CD 4

Créer le namespace : 4

Installer Argo CD : 4

Vérifier 4

6 STEP 2 — Accéder à Argo CD 4

1. Port-forward 4

2. Ouvrir navigateur 4

3. Récupérer le mot de passe 5

4. Login 5

🧠 **À comprendre ici** 5

7 STEP 3 — Créer le projet Git 6

📄 **Créer deployment.yaml** 6

📄 **Créer service.yaml** 6

8 STEP 4 — Push sur GitHub 7

1. Initialiser Git 7

2. Créer repo GitHub 7

3. Connecter repo 8

9 STEP 5 — Créer l'application Argo CD 8

📄 **Créer app.yaml** 8

10 STEP 6 — Déployer avec Argo CD 9

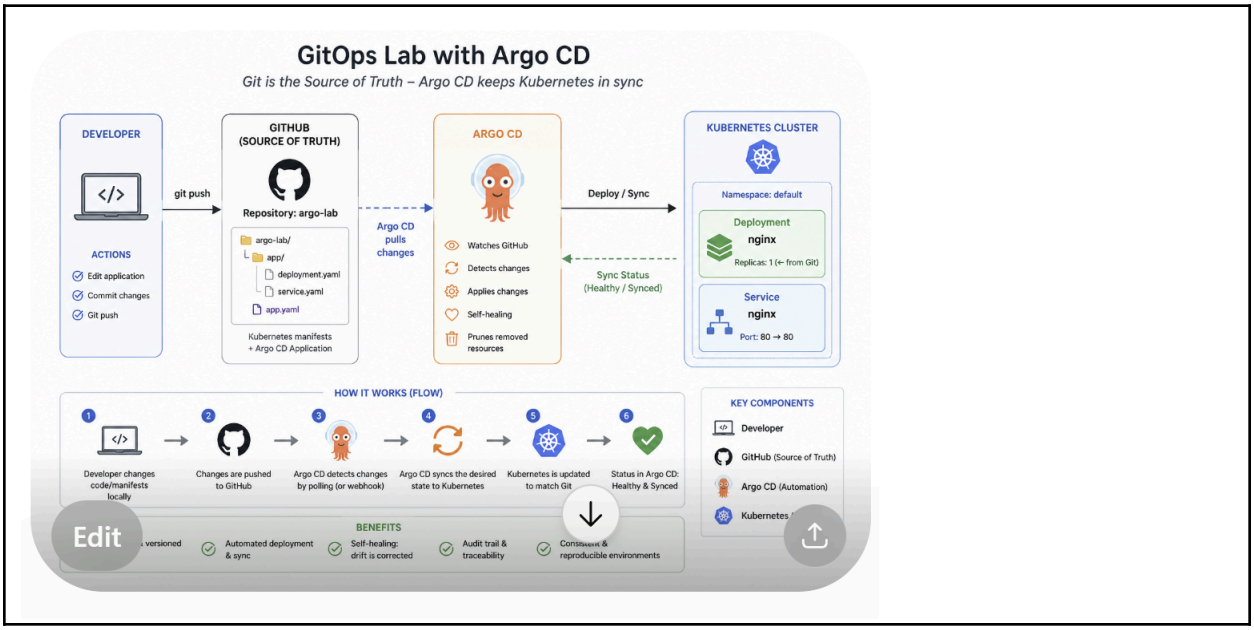
11 STEP 7 — Vérifier 9

12 STEP 8 — Vérifier dans UI Argo CD 10
13 STEP 9 — TEST IMPORTANT (GitOps réel) 11
1. Modifier le deployment 11
2. Push 11
3. Vérifier 11

---

1 Objectif

Déployer une application Kubernetes **uniquement via Git** grâce à Argo CD



---

2 Architecture

Votre PC → GitHub → Argo CD → Kubernetes

```
argo-lab/
├── app/
│   ├── deployment.yaml
│   └── service.yaml
└── app.yaml
```

---

3 PRÉREQUIS

On dois avoir :

- Kubernetes (k3d, minikube ou autre)
- kubectl installé
- Git installé
- Un compte GitHub
- Docker installé et démarré

---

4 STEP 0 — Verifier Kubernetes

👉 On dois voir au moins 1 node Ready

Dans le cas de notre lab on utilise k3d (pas minikube ou autre)

1. Démarrer K3d : k3d start
2. Voir les clusters actifs ou non , existants : k3d cluster list
3. Choisir un cluster (existant) et le démarrer : k3d cluster start name
4. Name = nom du cluster
5. Si pas de cluster existant , en créer un : k3d cluster create name
6. Puis saisir la commande : kubectl get nodes
7. On vérifiera qu'il existe au moins un nœud (node)
8. Ensuite basculer au step 1 :

Dans notre cas, on avait suite aux labs précédents plusieurs clusters. On a choisis le cluster qui s'intitule : henri, par exemple

On démarre ensuite ce cluster. Et il nous servira de cluster référence pour tout ce lab

```
henri@Henri:~/labs/argocd$ k3d cluster list
NAME          SERVERS   AGENTS   LOADBALANCER
gitops-lab    0/1       0/0      true
henri         0/1       0/0      true
ingress-lab   0/1       0/0      true
henri@Henri:~/labs/argocd$
henri@Henri:~/labs/argocd$ k3d cluster start henri
INFO[0000] Using the k3d-tools node to gather environment information
INFO[0000] Starting existing tools node k3d-henri-tools...
INFO[0000] Starting node 'k3d-henri-tools'
INFO[0000] Starting new tools node...
INFO[0001] Starting node 'k3d-henri-tools'
INFO[0002] Starting cluster 'henri'
INFO[0002] Starting servers...
INFO[0002] Starting node 'k3d-henri-server-0'
INFO[0005] All agents already running.
INFO[0005] Starting helpers...
INFO[0005] Starting node 'k3d-henri-tools'
INFO[0005] Starting node 'k3d-henri-serverlb'
INFO[0012] Injecting records for hostAliases (incl. host.k3d.internal) and for 3 network members into CoreDNS configmap.
..
INFO[0014] Started cluster 'henri'
henri@Henri:~/labs/argocd$
```

On vérifie la présence de un node : qui joue le rôle de control-plane master

```
henri@Henri:~/labs/argocd$
henri@Henri:~/labs/argocd$ kubectl get nodes
NAME                  STATUS   ROLES                  AGE   VERSION
k3d-henri-server-0    Ready    control-plane,master   36h   v1.31.5+k3s1
henri@Henri:~/labs/argocd$
henri@Henri:~/labs/argocd$
```

---

5 STEP 1 — Installer Argo CD

Créer le namespace :

kubectl create namespace argocd

Installer Argo CD :

kubectl apply -n argocd \
-f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

---

Vérifier

kubectl get pods -n argocd

👉 Attends que tout soit Running

```
henri@Henri:~/labs/argocd$
henri@Henri:~/labs/argocd$ kubectl get pods -n argocd
NAME                                                READY   STATUS            RESTARTS   AGE
argocd-application-controller-0                     0/1     Running           0          13s
argocd-applicationset-controller-654f78b44f-8kdw2   1/1     Running           0          13s
argocd-dex-server-bcf7845fd-2mvrd                   0/1     PodInitializing   0          13s
argocd-notifications-controller-5f944c679b-ll27d    1/1     Running           0          13s
argocd-redis-6879554f69-gd7lb                       1/1     Running           0          13s
argocd-repo-server-6b55bbdbd8-btjbh                 0/1     Running           0          13s
argocd-server-6b6b48974f-p7gfk                      0/1     Running           0          13s
henri@Henri:~/labs/argocd$
henri@Henri:~/labs/argocd$
```

---

6 STEP 2 — Accéder à Argo CD

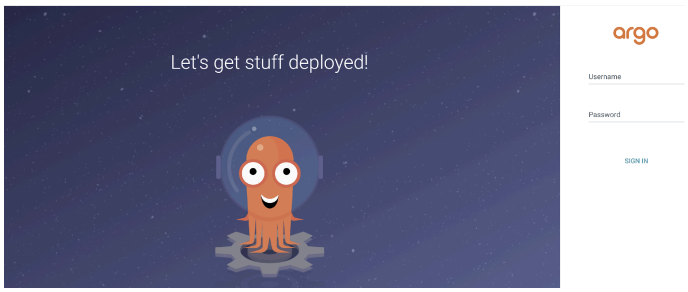
1. Port-forward

kubectl port-forward svc/argocd-server -n argocd 8080:443

---

2. Ouvrir navigateur

http://localhost:8080



---

3. Récupérer le mot de passe

kubectl get secret argocd-initial-admin-secret \
-n argocd -o jsonpath="{.data.password}" | base64 –decode

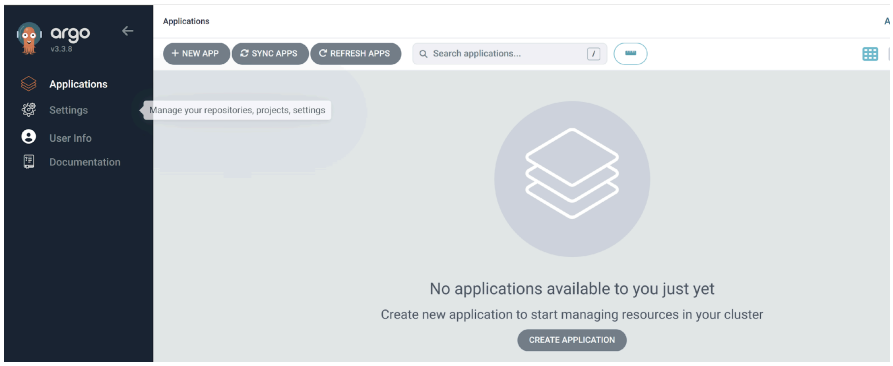
---

4. Login

- user : admin
- password : (commande ci-dessus)

🧠 **À comprendre ici**

👉 Argo CD est maintenant installé et prêt
👉 Il attend un repo Git



---

7 STEP 3 — Créer le projet Git

Sur ton PC :

mkdir argo-lab
cd argo-lab
mkdir app
cd app

```
henri@Henri:~/labs$ mkdir argo-lab
henri@Henri:~/labs$ cd argo-lab
henri@Henri:~/labs/argo-lab$ mkdir app
henri@Henri:~/labs/argo-lab$ cd app
henri@Henri:~/labs/argo-lab/app$ pwd
/home/henri/labs/argo-lab/app
henri@Henri:~/labs/argo-lab/app$
```

📄 **Créer deployment.yaml**

nano deployment.yaml

👉 colle :

apiVersion: apps/v1
kind: Deployment
metadata:

```
  name: nginx
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx
        ports:
        - containerPort: 80
```

📄 **Créer service.yaml**

nano service.yaml

👉 colle :

```
apiVersion: v1
kind: Service
metadata:
  name: nginx
spec:
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
```

---

8 STEP 4 — Push sur GitHub

1. Initialiser Git

```
cd ..
git init
git add .
git commit -m "initial commit"
```

```
henri@Henri:~/labs/argo-lab/app$ cd ..
henri@Henri:~/labs/argo-lab$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/henri/labs/argo-lab/.git/
henri@Henri:~/labs/argo-lab$ git add .
henri@Henri:~/labs/argo-lab$ git commit -m "initial commit"
[master (root-commit) efec71d] initial commit
 2 files changed, 29 insertions(+)
 create mode 100644 app/deployment.yaml
 create mode 100644 app/service.yaml
henri@Henri:~/labs/argo-lab$
```

---

2. Créer repo GitHub

👉 sur GitHub :
nom = argo-lab

Dans le cas de mon lab le repo crée est : https://github.com/guihen01/argo-lab

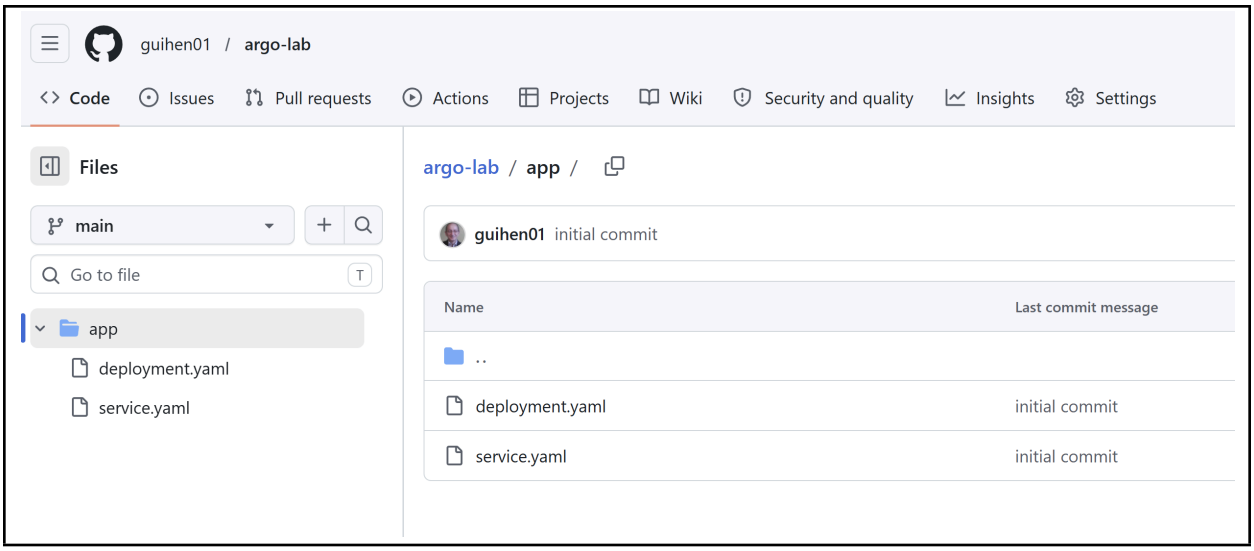
---

3. Connecter repo

git branch -M main
git remote add origin https://github.com/<your-username>/argo-lab.git
git push -u origin main

```
henri@Henri:~/labs/argo-lab$
henri@Henri:~/labs/argo-lab$ git branch -M main
henri@Henri:~/labs/argo-lab$ git remote add origin https://github.com/guihen01/argo-lab.git
henri@Henri:~/labs/argo-lab$ git push -u origin main
Username for 'https://github.com': guihen01
Password for 'https://guihen01@github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 567 bytes | 567.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/guihen01/argo-lab.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
henri@Henri:~/labs/argo-lab$
```

Resultat dasn github :



---

9 STEP 5 — Créer l'application Argo CD

📄 Créer app.yaml

nano app.yaml

👉 colle :

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: nginx-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/<your-username>/argo-lab
    targetRevision: main
    path: app
  destination:
    server: https://kubernetes.default.svc
    namespace: default
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

---

10 STEP 6 — Déployer avec Argo CD

👉 important

kubectl apply -f app.yaml

```
henri@Henri:~/labs/argo-lab$ kubectl apply -f app.yaml
application.argoproj.io/nginx-app created
henri@Henri:~/labs/argo-lab$
henri@Henri:~/labs/argo-lab$
```

---

11 STEP 7 — Vérifier

kubectl get pods

👉 Résultat attendu :

nginx-xxxxx Running

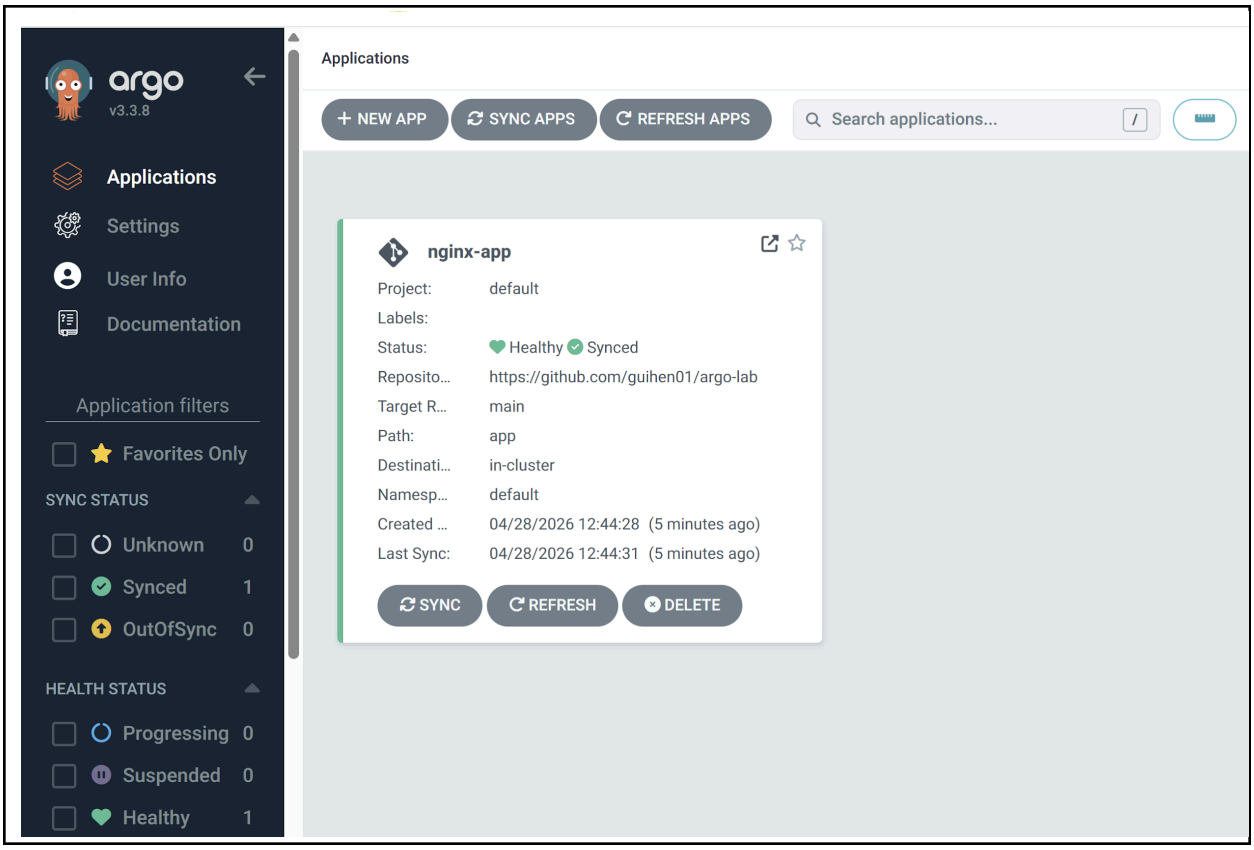
```
henri@Henri:~/labs/argo-lab$
henri@Henri:~/labs/argo-lab$ kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
demo-app-58bf5cfc8d-7ssp8     1/1     Running   0          18h
nginx-7769f8f85b-gxvzb        1/1     Running   0          54s
henri@Henri:~/labs/argo-lab$
henri@Henri:~/labs/argo-lab$
henri@Henri:~/labs/argo-lab$
```

---

12 STEP 8 — Vérifier dans UI Argo CD

Dans l'interface :

- Status → **Synced**
- Health → **Healthy**



---

13 STEP 9 — TEST IMPORTANT (GitOps réel)

1. Modifier le deployment

nano app/deployment.yaml

Change :

replicas: 3

---

2. Push

```
git add .
git commit -m "scale to 3"
git push
```

```
henri@Henri:~/labs/argo-lab$
henri@Henri:~/labs/argo-lab$ git add .
henri@Henri:~/labs/argo-lab$ git commit -m "scale to 3"
[main d1378e9] scale to 3
 2 files changed, 19 insertions(+), 1 deletion(-)
 create mode 100644 app.yaml
henri@Henri:~/labs/argo-lab$ git push
Username for 'https://github.com': guihen01
Password for 'https://guihen01@github.com':
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 16 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 630 bytes | 315.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/guihen01/argo-lab.git
   efec71d..d1378e9  main -> main
henri@Henri:~/labs/argo-lab$
henri@Henri:~/labs/argo-lab$
```

---

3. Vérifier

kubectl get pods

👉 Résultat :

3 pods nginx

```
henri@Henri:~/labs/argo-lab$
henri@Henri:~/labs/argo-lab$ kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
demo-app-58bf5cfc8d-7ssp8     1/1     Running   0          18h
nginx-7769f8f85b-44nqg        1/1     Running   0          25s
nginx-7769f8f85b-gxvzb        1/1     Running   0          12m
nginx-7769f8f85b-ln9mc        1/1     Running   0          25s
henri@Henri:~/labs/argo-lab$
```

Sans kubectl apply → Argo CD